מערכות הפעלה - 20594

סמסטר 2020א

מרכז ההוראה: דוד שריאל

התחיל ב:	10:00 ,25/06/2020
מצב	הסתיים
הושלם ב-	13:02 ,25/06/2020
הזמן שלקח	3 שעות 1 דקה

מידע

בחינה בעקרונות מערכות הפעלה

קראו בעיון לפני שתתחילו בפתרון הבחינה!

1. **המבחן מורכב משלושה חלקים.**
2. **בחלקים א' ו ב' מופיעות שאלות פתוחות. את התשובות לכל השאלות הפתוחות יש לכתוב בתיבת טקסט .ענו תשובות מלאות בקיצור נמרץ. אין חובה להשתמש בכל השורות המוקצות לצורך התשובות. אין לחרוג מהמקום המוקצה.**
3. **בחלק ג' (שאלות אמריקאיות) עליכם לבחור בכל פעם בתשובה יחידה מבין התשובות המוצעות ולהקיף בעיגול את אות התשובה שבחרתם.**
4. **משך הבחינה - שלוש שעות.**

בהצלחה!

שאלה **1**

הושלם

ניקוד השאלה: 20.00

בסעיפים א,ב בשאלה זו עליכם להוכיח או להפריך את הטענות המופיעות לגבי אלגוריתם הבנקאי (banker algorithm). בסעיפים ג,ד, עליכם לענות על השאלות.

א. (6 נקודות)
אם אלגוריתם הבנקאי גילה שהמערכת נמצאת במצב לא בטוח, אז בהכרח שהמערכת תגיע לקיפאון בעתיד.

ב. (6 נקודות)
אם אלגוריתם הבנקאי גילה שהמערכת נמצאת במצב בטוח, אנחנו יודעים בוודאות שלא יתכן קיפאון בעתיד בהשתתפות המשאבים עליהם הורץ אלגוריתם.

ג. (5 נקודות)
מהן שתי המגבלות העיקריות של אלגוריתם הבנקאי אשר הופכות אותו לבלתי שימושי באופן מעשי? הניחו שמדובר על תהליכים שתופסים משאבים לאורך תקופה ארוכה (תקופה של שעות או של ימים).

ד. (3 נקודות)
הביאו דוגמא לשימוש באלגוריתם הבנקאי במערכת הפעלה המנהלת נורות בבית חכם. לגוגמא הניחו שקיימים 2 סוגי נורות: נורות מחיפות צבעים ונורות עם אור בצבע קבוע.

מותר גם לתת לדימיון להתפרע ולא להיות מוגבלים לנורות ו/או לשני סוגים בלבד.

א. טענה זו **אינה נכונה**. נפריך אותה - גם במצב בו אלגוריתם הבנקאי גילה שהמערכת נמצאת במצב לא בטוח עדיין ייתכן מצב בו לא כל התהליכים ייבקשו את מספר המשאבים המקסימלי עבורם וכך זה לא הכרח שהמערכת תגיע לקיפאון בעתיד.

ב. טענה זו הינה **נכונה**. נוכיח אותה - במצב בו אלגוריתם הבנקאי גילה שהמערכת נמצאת במצב בטוח והמשאבים יינתנו לפי המסלול הבטוח שחושב ע"י האלוגריתם (ובהכרח קיים כזה משום שהאלגוריתם גילה שהמערכת נמצאת במצב בטוח) - במצב לא לא ייתכן קיפאון בעתיד. לעומת זאת, אם המשאבים לא יינתנו לפי המסלול הבטוח שחושב, ייתכן מצב של קיפאון בעתיד.

ג. שתי המגבלות העיקריות של אלגוריתם הבנקאי אשר הופכות אותו לבלתי שימושי באופן מעשי הינן:

1. האלגוריתם דורש לדעת מראש לגבי כל ההליכים מה מספר המשאבים המקסימלי שהם עלולים לדרוש - דבר שלא תמיד אפשרי ודורש הרבה מקום.
2. כאשר האלגוריתם מחשב מצב לא בטוח הוא מסרב לבקשות של תהליכים ומצב זה יכול להימשך הרבה זמן עד שאותן בקשות לא יגרמו לחישוב של מצב לא בטוח - יכול לגרום להרעבה.

ד. נביא דוגמא לשימוש באלגוריתם הבנקאי במערכת הפעלה המנהלת נורות בבית חכם:

נניח עבור הדוגמה כי הנורות הם התהליכים שצורכים משאבים, כאשר המשאב הוא זרם חשמל ולכל נורה יש צריכה מקסימלית שונה, כלומר, נורות בצבע אור קבוע צורכות זרם חשמלי קבוע (או כלום אם הן כבויות) ונורות מחליפות צבעים יכולות לצרוך זרם חשמל משתנה (כאשר הזרם המקסימלי נקבע לפי הצבע שדורש הכי הרבה זרם).

תחילה האלגוריתם מקבל את המידע לגבי צריכה החשמל המקסימלית מכל הנורות.

לאחר מכן, הוא מקבל את בקשות הצריכה מהנורות (האם הן כבויות או דלוקות, ואם הן דולקות כמה זרם חשמל הן דורשות), האלגוריתם מחשב את האם קיים מצב בטוח או לא לחלוקה של זרם החשמל בין כל הנורות.

במידה וקיים: ייחלק את המשאבים לפי החלוקה שחושבה.

במידה ולא קיים: ייכבה את חלק מהנורות (כלומר, לא ייספק להם זרם חשמל) על-מנת לא להגיע למצב של נפילת חשמל (מכיוון שחולקו יותר משאבים ממה שקיים)

שאלה **2**

הושלם

ניקוד השאלה: 20.00

ביל וסטיב החליטו לחבל בקוד הפתוח של מערכות Linux באמצעות הכנסת זמן שהיה גרוע ככל האפשר למערכת. בשאלה זו נעזור להם לתכנן זמן גרוע ככל האפשר.

ביל הציע לממש את מדיניות הזימון LJF–Longest Job First. מדיניות זו הפוכה בהתנהגותה ממדיניות SJF שלמדנו. כלומר, הניחו שכל התהליכים מגיעים יחד וזמני הביצוע ידועים מראש, ונריץ את התהליכים לפי זמן הביצוע בסדר יורד, כשכל תהליך רץ מתחילתו ועד סופו.

א. (7 נקודות)
האם מדיניות LJF היא הגרועה ביותר מבין כל המדיניות בלי הפקעה, לפי מדד זמן השהיה הממוצע? הוכיחו או תנו דוגמה נגדית.

ב. (8 נקודות)
סטיב טוען שקיימות מדיניות עם הפקעה גרועות יותר מ-LJF. האם סטיב צודק? הסבירו.אם כן, תנו דוגמה.

ג. (5 נקודות)
נניח שהגיעו למערכת n תהליכים. נסמן את זמני הריצה שלהם ב t_i ואת זמני השהיה שלהם תחת SJF ב S_i. (שימו לב, זמן השהיה כולל את זמן הריצה). מהו זמן ההשהיה של התהליך ה- i תחת LJF? הניחו שזמני הריצה של כל התהליכים שונים.

א. מדיניות LJF הינה ההגרועה ביותר מבין כל המדיניות בלי הפקעה (עפ"י מדד זמן השהייה הממוצע) וזאת משום שבעת בחירת התהליך הראשון לביצוע אנו בוחרים את התהליך עם זמן הביצוע הארוך ביותר (נסמנו t1) אזי זמן השהייה של התהליך ההראשון יהיה הזמן המקסימלי האפשרי t1, כאשר נבחר את התהליך השניה לביצוע אנו נבחר את התהליך עם זמן הביצוע השני הכי ארוך (נסמנו t2) ובכך זמן השהייה של התהליך השני יהיה הזמן המקסימלי האפשרי t1 + t2 וכך הלאה..
סה"כ זמן השהייה הממוצע יהיה מקסימלי ושווה ל-
(t1 + (t1+t2) + ... + (t1 + t2 + ... + tn)

מקווה שהנוסחה רשומה ברור.

ב. סטיב צודק בטענתו - אכן קיימת מדיניות עם הפקעה אשר זמן השהייה הממוצע בה גרוע יותר מאשר ב-LJF - לדוגמה - מדיניות בה נפקע תהליך כאשר הוא מגיע לקוואנטום האחרון שלו ובכך כל התהליך ייאלץ לחכות שכל

התהליכים ייסיימו את כל זמן הביצוע שלהם (פחות הקוואנטום האחרון) ורק לאחר-מכן כל התהליכים יסיימו את הביצוע שלהם. במצב זה זמן השהייה של כל תהליך הינו סדר גודל של סכום זמני הביצוע של כל התהליכים יחד.

ג. אם n התהליכים שהגיעו למערכת בוצעו תחת מדיניות הזימון SJF באופן הבא:

$$SJF: t1 + (t1+t2) + \ldots + (t1+t2+\ldots+tn)$$

כלומר, t1 הינו הקצר ביותר ו-tn הינו הארוך ביותר, אזי זמן השהייה של התהליך ה-i תחת מדיניות הזימון LJF הינה:

$$LJF_i = S_n - S_i + t_i$$

כאשר

$$S_m = \sum_{i=1}^{m} t_i$$

שאלה **3**

הושלם

ניקוד השאלה: 15.00

א. (7 נקודות)
הסבירו מהי הבעייתיות של האלגוריתם LRU להחלפת דפים כאשר מממשים אותו באמצעות רשימה משורשרת.

ב. (8 נקודות)
רשמו במשבצת המיועדת לתשובות רשימת התיחסויות (לדפי זיכרון) המדגימה שלעתים
אלגוריתם LRU יכול לגרום לכמות גדולה יותר של פסיקות דף למרות שלתהליך מקצים יותר מסגרות זיכרון פיזיות, או, לחילופין, נמקו מדוע הדבר בלתי אפשרי.

א. הבעייתיות של האלגוריתם LRU להחלפת דפים כאשר ממשים אותו באמצעות רשימה משורשרת הינה שבכל פעם שהאלגוריתם צריך לחשב כמה זמן עבר מאז השימוש האחרון עבור כל הדפים ברשימה עליו לעבור על כל הרשימה המשורשרת כדי להחליף הצבעות בין הדפים. במימוש זה הגישה לזיכרון דורשת הרבה זמן והופכת את פעולת האלגוריתם לאיטית.

ב. דבר זה הינו בלתי אפשרי. נוכיח זאת בעזרת אינדוקציה:
בכל הקריאות הראשונות בהכרח מתבצעת פסיקה בשני המקרים וזאת משום שמסגרות הזיכרון הפיזיות ריקות.
נניח כי לאחר רצף גישות מסויים מספר הפסיקות בשתי האפשרויות הינה זהה.
נוכיח כי עבור קריאת דף x כלשהו עדיין מספר הפסיקות בשתי האפשרויות יהיה זהה או לטובת האפשרות עם מספר המסגרות הגדול יותר:
נניח כי כבר קראו לדף x ב-n הקריאות האחרונות (כאשר n הוא מספר המסגרות הקטן יותר) אזי הוא בהכרח קיים גם באחת המסגרות באפשרות עם הזיכרון הגדול יותר, ואילו אם לא קראו לדף ב-n הקריאות האחרונות אזי תתקיים פסיקת דף באפשרות עם מספר המשגרות הקטן יותר ואילו באפשרות השנייה ייתכן שלא (תלוי ב-N המסגרות שאחרי המסגרת ה-n)

שאלה **4**

הושלם

ניקוד השאלה: 5.00

הסבירו כיצד ניתן לזהות סחרור (threshing) במערכת.

כאשר המערכת נמצאת במצב של סחרור ישנן פסיקות דף לעיתים קרובות ובכך התהליך לא מצליח לבצע כמות רבה של פעולות בין לבין - לכן, ניתן לזהות סחרור במערכת על-ידי שימוש במונה של מספר פסיקות הדף אל מול התקדמות התהליך עצמו.
במצב בו המונה של מספר פסיקות הדף גדול משמעותית מהמונה של התקדמות התהליך עצמו אנו יכולים לדעת כי המערכת נמצאת במצב של סחרור.

שאלה **5**

הושלם

ניקוד השאלה: 5.00

לפניכם קטע קוד מתוך fs.c של מערכת הפעלה xv6:

```
217  // Copy a modified in-memory inode to disk.
218  // Must be called after every change to an ip->xxx field
219  // that lives on disk.
220  // Caller must hold ip->lock.
221  void
222  iupdate(struct inode *ip)
223  {
224    struct buf *bp;
225    struct dinode *dip;
226
227    bp = bread(ip->dev, IBLOCK(ip->inum, sb));
228    dip = (struct dinode*)bp->data + ip->inum%IPB;
229    dip->type = ip->type;
230    dip->major = ip->major;
231    dip->minor = ip->minor;
232    dip->nlink = ip->nlink;
233    dip->size = ip->size;
234    memmove(dip->addrs, ip->addrs, sizeof(ip->addrs));
235    log_write(bp);
236    brelse(bp);
237  }
```

הסיקו מתוך הקוד איזה סוג של i-node cache קיים במערכת: write through או non write through?

מתוך הקוד הנ"ל ניתן להסיק כי סוג ה-i-node-cache אשר קיים במערכת הינו write through וזאת משום שבשורה 234 מתבצעת הפקודה memmove אשר מעתיקה את הבתים מהמקור ליעד - כלומר, מתבצעת כתיבה ישירה של המידע.

שאלה **6**

הושלם

ניקוד השאלה: 5.00

לפניכם קטע קוד מתוך shell.c. הסבירו מדוע מתעלמים מכל אחד מן הסיגנלים המופיעים בשורות 518-522.

נא להקפיד להשיב בפורמט 'משפט לכל סיגנל'.

```
495 int main(int argc, char ** argv) {
496     char command[MAX_COMMAND_LEN + 1];
497     char * nextCommand = NULL;
498     struct jobSet jobList = { NULL, NULL };
499     struct job newJob;
500     FILE * input = stdin;
501     int i;
502     int status;
503     int inBg;
504
505     if (argc > 2) {
506         fprintf(stderr, "unexpected arguments; usage: "
507                         "<commands>\n");
508         exit(1);
509     } else if (argc == 2) {
510         input = fopen(argv[1], "r");
511         if (!input) {
512             perror("fopen");
513             exit(1);
514         }
515     }
516
517
518     signal (SIGINT, SIG_IGN);
519     signal (SIGQUIT, SIG_IGN);
520     signal (SIGTSTP, SIG_IGN);
521     signal (SIGTTIN, SIG_IGN);
522     signal (SIGTTOU, SIG_IGN);
523
524     while (1) {
525         if (!jobList.fg) {
526             /* no job is in the foreground */
527
528             /* see if any background processes have exited */
529             checkJobs(&jobList);
```

SIGINT - SIGnal Interrupt - שורה זו מונעת מ-interrupts לעצור את התהליך.

SIGQUIT - SIGnal Quit - שורה זו מונעת מביטול התהליך.

SIGTSTP - Interactive Stop SIGnal - שורה זו מונעת מעצירת התהליך ע"י המשתמש.

SIGTTIN - SIGnal TeleType INput - שורה זו מונעת מהתהליך להתנתק מהטרמינל (למשל בעת הקלדה)

SIGTTOU - SIGnal TeleType OUtput - שורה זו מונעת מהתהליך להתנתק מהטרמינל (למשל מפקודה חיצונית מטרמינל אחר)

מתעלמים מהסיגנלים הנ"ל על-מנת לא להפריע לתהליך שמתחיל משורה 524.

שאלה **7**

הושלם

ניקוד השאלה: 5.00

לפניכם קטע קוד מקובץ uthread.c של מערכת הפעלה xv6:
הסבירו מדוע מחסרים 32 מ t->sp בשורה 74 ופרטו מה בדיוק נשמר ב 32 סיביות.

```
63  void
64  thread_create(void (*func)())
65  {
66    thread_p t;
67
68    for (t = all_thread; t < all_thread + MAX_THREAD; t++) {
69      if (t->state == FREE) break;
70    }
71    t->sp = (int) (t->stack + STACK_SIZE);
72    t->sp -= 4;
73    * (int *) (t->sp) = (int)func;
74    t->sp -= 32;
75    t->state = RUNNABLE;
76  }
```

מחסרים 32 בתים מ-t->sp על-מנת לפנות מקום ל-8 אוגרים - כלומר, בכל 4 בתים נשמר אוגר אחר. אוגרים אלו ישמשו לאחר-מכן את thread_switch.

שאלה **8**

הושלם

ניקוד השאלה: 5.00

נתון קטע קוד הבא:

באיזו שורה תבצע מערכת הפעלה לינוקס הקצאת מסגרת פיזית עבור array?

```
1  int array[4096];
2
3  int main () {
4      int x = 0;
5      int y = array[0];
6      int z = array[1];
7      array[2] = x+y+z;
8  …
9  }
```

תניחו שאין שום משתנים גלובליים נוספים חוץ מ-array

מערכת ההפעלה לינקוס תבצע את הקצאת המסגרת הפיזית עבור array בתחילת הפונקציה main כלומר, בשורה 3 כאשר הפרצדורה main מאותחלת.

שאלה **9**

הושלם

ניקוד השאלה: 5.00

מהו החיסרון המובהק של שיטת ה- rendezvous לעומת ה- mailbox ב-message passing?

יש לבחור תשובה אחת:

- **שיטת ה-rendezvous קשה יותר למימוש במערכת הפעלה.**
- **בשיטת ה-rendezvous לא ניתן לממש מניעה הדדית בגישה לקטעים קריטיים (critical sections).**
- **שיטת ה-rendezvous פחות גמישה מכיוון שהתהליכים (השולח והנמען) הופכים להיות מסונכרנים (תלויים זה בזה).**
- **שיטת ה-rendezvous גורמת לסחרור (thrashing)**

שאלה **10**

הושלם

ניקוד השאלה: 5.00

בחרו סיגנל (signal) אשר אי-אפשר להתעלם ממנו (באמצעות SIG_IGN):

יש לבחור תשובה אחת:

- **SIGSEGV**
- **SIGINT**
- **SIGKILL**
- **SIGALRM**

שאלה **11**

הושלם

ניקוד השאלה: 5.00

מערכת ההפעלה יוניקס מנסה לתת לתהליכים בעלי עדיפות גבוהה timeslice ארוך, ואילו לתהליכים בעלי עדיפות נמוכה מקצה timeslice קצר. תהליכים בעלי עדיפות גבוהה הם בדרך-כלל התהליכים המבצעים פעולות קלט/פלט (IO bounded), ואילו תהליכים בעלי עדיפות נמוכה הם התהליכים המבצעים חישובים (CPU bounded). כתוצאה מכך, במקרים שכל התהליכים במערכת יוניקס הם תהליכים חישוביים, יקרו הרבה החלפות תוכן (context switches). כמובן שהדבר לא רצוי, מכיוון שהיינו רוצים לתת לתהליכים חישוביים לרוץ כמה שיותר זמן וללא החלפות תוכן תדירות. בחרו שיטת תזמון הפותרת בעיה זו:

יש לבחור תשובה אחת:

- **shortest remaining time next**
- **shortest job first**
- **fair share scheduling**
- **first come first served**

שאלה **12**

הושלם

ניקוד השאלה: 5.00

איזו פעולה מן הפעולות הבאות אפשר לבצע אך ורק במצב ראשוני (kernel mode) במערכת ההפעלה Linux?

יש לבחור תשובה אחת:

- **חסימת פסיקות החומרה (disabling hardware interrupts)**
- **החלפת תהליכונים (thread switch) כאשר מדובר בספריית תהליכונים ברמת המשתמש**
- **השמת ערך במשתנה גלובאלי**
- **את כל שלוש הפעולות הנ"ל יש לאפשר אך ורק במצב ראשוני**